[ADR] Encrypted Data Access and Key Delegation

Status

Proposed by @Sergey Poluyan @Ulad Palinski

Stakeholders: @Fred Jin

Outcome

Status: planned

Executive Summary

This ADR extends the existing DDC authorization system to support **encrypted data storage** with **delegated key access** for AI agents. It builds upon the recursive token chain model documented in [ADR] Authentication and authorization , adding encryption primitives that enable secure data sharing between users, data services, and AI agents.

Key Goals:

- Users can upload encrypted data to DDC
- Encryption keys can be delegated to data services and AI agents
- Access is verifiable and auditable
- Keys can be added/revoked for existing data
- Same authorization pattern applies to Storage and Compute layers

---

Context

Concept mapping

Concept	AWS Equivalent	DDC Concept
Identity	IAM User / Role	**Wallet Address (Ed25519)**
Authorization	IAM Policy	**AuthToken (Signed JWT-like)**
Encryption Key	KMS Key	**DEK (AES-256) wrapped in an EncryptionGrant**
Key Manager	AWS KMS Service	**KES (Key Escrow Service) or Embedded**

DDC Infrastructure Layers

The DDC system consists of two primary layers:

Layer	Description	Distribution Model
Storage	Decentralized content-addressed storage	Fully decentralized (blockchain-verified)
Compute	AI agent execution runtime	Distributed (orchestrator-managed)

Current Data Flow (Unencrypted)

```
flowchart LR
    subgraph Current["Current: Plaintext"]
        User1[User] -->|plaintext| Node[DDC Node]
        Node <-->|plaintext| Agent1[Agent]
    end

    User1 -.->|AuthToken| Node
    Node -.->|Token Verification| Node

    style User1 fill:#ffcccc
    style Node fill:#ffcccc
    style Agent1 fill:#ffcccc
```

Problem: Data is stored and transmitted in plaintext. Storage nodes and network intermediaries can read user data.

Target Data Flow (Encrypted)

```
flowchart LR
    subgraph Target["Target: Encrypted"]
        User2[User] -->|ciphertext| Node[DDC Node]
        Node <-->|ciphertext| Agent2[Agent]
    end

    User2 -.->|AuthToken + EncryptionGrant| Node
    Node -.->|Token + Grant Verification| Node
    Agent2 -.->|DEK via Grant / Decrypt in V8| Node

    style User2 fill:#ccffcc
    style Node fill:#ccffcc
    style Agent2 fill:#ccffcc
```

Solution: Data encrypted client-side with DEK (Data Encryption Key). DEK shared via delegated encryption grants.

Current Authorization System

Recursive Token Chain

The DDC uses a **recursive JWT-like token chain** where each token delegates a subset of permissions to the next:

```
flowchart TB
    subgraph TokenChain["Token Chain Structure"]
        Root["🔐 ROOT TOKEN<br/>(Bucket Owner)<br/>subject: 0
xabc"]
        Delegated["📜 DELEGATED TOKEN<br/>(Agent Service)<br/
>subject: 0xdef"]
        Final["✅ FINAL TOKEN<br/>(Agent)<br/>subject: empt
y"]

        Root -->|prev| Delegated
```

```
        Delegated -->|prev| Final
    end

    Root -.- Note1["Signed by bucket owner<br/>(on-chain veri
fied)"]
    Root -.- Note2["Grants access to Agent Service"]
    Delegated -.- Note3["Signed by Agent Service (0xabc)"]
    Delegated -.- Note4["Grants access to specific agent"]
    Final -.- Note5["Signed by Agent (0xdef)"]
    Final -.- Note6["Terminal token<br/>(no further delegatio
n)"]

    style Root fill:#e1f5fe
    style Delegated fill:#fff3e0
    style Final fill:#e8f5e9
```

Current Token Specification

```
message Signature {
  Algorithm algorithm = 1;  // ED_25519 or SR_25519
  bytes signer = 2;         // Public key of signer
  bytes value = 3;          // Signature bytes
}

message AuthToken {
  Signature signature = 1;
  Payload payload = 2;
}

message Payload {
  optional AuthToken prev = 1;      // Previous token in chai
n
  optional bytes subject = 2;       // Recipient public key
  optional bool canDelegate = 3;    // Can subject further de
legate?
```

```
  optional uint64 bucketId = 4;     // Target bucket
  repeated Operation operations = 5; // Allowed operations
  optional int64 expiresAt = 6;     // Expiration timestamp
  optional bytes pieceCid = 7;      // Specific piece access
}

enum Operation {
  UNKNOWN = 0;
  PUT = 1;
  GET = 2;
  DELETE = 3;
}
```

Token Verification Flow

```
// Simplified verification in storage node
func (g *gater) expandAndVerifyToken(token *AuthToken, expand
ed *ExpandedToken, depth int) error {
    // 1. Verify signature
    if !verifySignature(token) {
        return ErrInvalidSignature
    }

    // 2. Check expiration
    if token.IsExpired() {
        return ErrExpired
    }

    // 3. Validate expiration chain (child <= parent)
    if expanded.expiresAt != 0 && expanded.expiresAt > token.
Payload.ExpiresAt {
        return ErrInvalidExpirationTime
    }

    // 4. Validate permissions (child ⊆ parent)
```

```
    if !permissionsValid(expanded.operations, token.Payload.O
perations) {
        return ErrInvalidPermissions
    }

    // 5. Validate issuer matches previous subject
    if len(expanded.rootIssuer) > 0 && !bytes.Equal(expanded.
rootIssuer, token.Payload.Subject) {
        return ErrInvalidIssuer
    }

    // 6. Recurse to parent token
    if token.Payload.Prev != nil {
        return g.expandAndVerifyToken(token.Payload.Prev, exp
anded, depth+1)
    }

    // 7. Root token: verify bucket ownership on-chain
    return g.verifyBucketOwnership(token.Signature.Signer, ex
panded.bucketID)
}
```

Current Limitations

Aspect	Current State	Limitation
Data Privacy	Plaintext storage	Storage nodes can read data
Key Management	None	No encryption key delegation
Agent Access	Token-based	No secure key delivery to agents

Problem Statement

Primary Use Case: Secure AI Agent Data Access

```
flowchart TB
    subgraph UseCase["🎯 Use Case: Secure AI Agent Data Acces
s"]
        Step1["1️⃣ User 0x123 has sensitive data"]
        Step2["2️⃣ User delegates write access to Agent Servic
e 0xabc"]
        Step3["3️⃣ Agent Service deploys AI agents that need t
o process user data"]
        Step4["4️⃣ Agents should only access data with user's
permission"]
        Step5["5️⃣ Keys must expire and be revocable"]

        Step1 --> Step2 --> Step3 --> Step4 --> Step5
    end

    style Step1 fill:#e3f2fd
    style Step2 fill:#fff3e0
    style Step3 fill:#f3e5f5
    style Step4 fill:#e8f5e9
    style Step5 fill:#ffcdd2
```

Challenges

1. **Client-Side Encryption**: Users must encrypt data before upload
2. **Key Delegation**: DEK must be securely shared with authorized parties
3. **Hierarchical Access**: User → Data Service → Agent hierarchy
4. **Revocability**: Must be able to revoke access to existing encrypted data
5. **Verification**: Storage and compute nodes must verify encryption grants
6. **Uniformity**: Same pattern for storage API and compute context API

Requirements

Functional Requirements

Requirement	Priority
Users can upload encrypted data to DDC	Must Have
Users can grant encryption key access to data services	Must Have
Data services can derive time-limited keys for agents	Must Have
Users can add grants to existing encrypted data	Must Have
Users can revoke grants for existing encrypted data	Must Have
Agents access encrypted data via context API	Must Have
Storage nodes verify encryption grants	Should Have
All key access is auditable	Should Have
Keys auto-expire based on grant constraints	Must Have
Same pattern works for storage and compute	Must Have

Non-Functional Requirements

Requirement	Metric
Key derivation latency	< 10ms
Grant verification latency	< 5ms
Revocation propagation	< 1 minute
Encryption overhead	< 5% of data size
No single point of failure for key access	99.9% availability

Design Decisions

1. Extend AuthToken (vs. Separate System)

Decision: Extend the existing `AuthToken` with encryption grant fields.

Rationale:

- Maintains single authorization primitive
- Leverages existing token verification infrastructure
- Backward compatible (encryption grant is optional)

- Consistent developer experience

Alternatives Considered:

- Separate `EncryptionToken`: Rejected due to complexity of managing two token types
- On-chain encryption registry: Rejected due to latency and cost

2. Hybrid Key Management

Decision: Support both **embedded DEK** (in token) and **key reference** (via Key Escrow Service).

Rationale:

- Embedded DEK: Simple, offline, suitable for short-lived grants
- Key reference: Enables revocation, suitable for long-lived grants
- Applications choose based on their requirements

3. HKDF for Key Derivation

Decision: Use HKDF-SHA256 for deriving agent keys from service keys.

Rationale:

- Industry standard (RFC 5869)
- Deterministic: Same input produces same output
- Supports context binding (agent ID, nonce)
- Compatible with WebCrypto API (runs in V8)

4. AES-256-GCM for Data Encryption

Decision: Use AES-256-GCM for symmetric encryption.

Rationale:

- Authenticated encryption (confidentiality + integrity)
- Hardware acceleration available
- WebCrypto API support

- Industry standard for data at rest

5. X25519 for Key Exchange

Decision: Use X25519 (Curve25519 ECDH) for encrypting DEK to recipients.

Rationale:

- Compatible with Ed25519 keys (can derive X25519 from Ed25519)
- Fast and secure
- Widely supported

Architecture

System Overview

```
flowchart TB
    subgraph User["👤 USER (0x123)"]
        direction LR
        Ed25519["🔑 Ed25519 Key<br/>(Signing)"]
        X25519["🔐 X25519 Key<br/>(Encryption)"]
        KeyMgr["📋 Key Manager<br/>• Generate DEK<br/>• Encry
pt data with DEK<br/>• Create EncryptionGrants<br/>• Manage g
rant lifecycle"]
    end

    subgraph AgentService["🏢 AGENT SERVICE (0xabc)"]
        subgraph KDS["Key Derivation Service"]
            direction LR
            Grant["User's Grant"] --> DecryptDEK["Decrypt DE
K"]
            DecryptDEK --> DeriveKeys["Derive Agent Keys"]
            DeriveKeys --> Wrap["Wrap"]
            DecryptDEK --> ServiceDEK["Service DEK"]
            DeriveKeys --> AgentDEKs["Agent-specific DEKs"]
        end
```

```
    end

    subgraph Agents["🤖 AI AGENTS (V8 Isolates)"]
        direction LR
        Alpha["Agent Alpha<br/>HKDF(svc, alpha)<br/>context.e
ncryption.decrypt(...)"]
        Beta["Agent Beta<br/>HKDF(svc, beta)<br/>context.encr
yption.decrypt(...)"]
        Gamma["Agent Gamma<br/>HKDF(svc, gamma)<br/>context.e
ncryption.decrypt(...)"]
    end

    User -->|"AuthToken + EncryptionGrant"| AgentService
    AgentService -->|"Derived Tokens + Wrapped DEKs"| Agents

    style User fill:#e3f2fd
    style AgentService fill:#fff8e1
    style Agents fill:#e8f5e9
    style Alpha fill:#c8e6c9
    style Beta fill:#c8e6c9
    style Gamma fill:#c8e6c9
```

Key Hierarchy

```
flowchart TB
    MasterKey["🔑 User Master Key<br/>(Ed25519 Keypair)"]
    DEK["🔐 Data Encryption Key<br/>(AES-256, per-piece)"]

    MasterKey -->|Generates| DEK

    DEK -->|"ECDH Wrap (X25519)"| WrappedA["📦 Wrapped DEK fo
r<br/>Agent Service A (0xabc)"]
    DEK -->|"ECDH Wrap (X25519)"| WrappedB["📦 Wrapped DEK fo
r<br/>Agent Service B (0xdef)"]
```

```
    WrappedA -->|HKDF Derivation| Agent1["🤖 Agent 1 Key"]
    WrappedA -->|HKDF Derivation| Agent2["🤖 Agent 2 Key"]
    WrappedA -->|HKDF Derivation| Agent3["🤖 Agent 3 Key"]

    style MasterKey fill:#e1f5fe
    style DEK fill:#fff3e0
    style WrappedA fill:#f3e5f5
    style WrappedB fill:#f3e5f5
    style Agent1 fill:#e8f5e9
    style Agent2 fill:#e8f5e9
    style Agent3 fill:#e8f5e9
```

Encryption Grant Flow

```
sequenceDiagram
    autonumber
    participant User as 👤 User (0x123)
    participant DDC as 💾 DDC Node
    participant Service as 🏢 Agent Service (0xabc)
    participant Agent as 🤖 AI Agent

    rect rgb(227, 242, 253)
        Note over User: 1. USER CREATES ENCRYPTED DATA
        User->>User: Generate DEK (AES-256)<br/>dek = crypto.
randomBytes(32)
        User->>User: Encrypt Data<br/>ciphertext = AES-GCM(pl
aintext, dek, nonce)
        User->>DDC: Upload ciphertext<br/>cid = ddc.store(buc
ketId, ciphertext)
        User->>User: Create EncryptionGrant<br/>ephemeralKey
= X25519.generate()<br/>sharedSecret = X25519.ecdh(ephemeralK
ey, servicePubKey)<br/>wrappedDEK = AES-GCM(dek, sharedSecre
t)
    end
```

```
    rect rgb(255, 243, 224)
        Note over User,Service: 2. USER DELEGATES TO AGENT SE
RVICE
        User->>Service: AuthToken + EncryptionGrant<br/>subje
ct: 0xabc, bucketId: 123<br/>operations: [GET], canDelegate:
true<br/>expiresAt: now + 7 days<br/>constraints: maxDepth=1,
maxTTL=86400
    end

    rect rgb(243, 229, 245)
        Note over Service: 3. AGENT SERVICE DERIVES AGENT KEY
S
        Service->>Service: Decrypt DEK from Grant<br/>sharedS
ecret = X25519.ecdh(serviceKey, ephemeralPubKey)<br/>dek = AE
S-GCM.decrypt(encryptedDataKey, sharedSecret)
        Service->>Service: Derive Agent-Specific Key<br/>agen
tNonce = crypto.randomBytes(16)<br/>agentDEK = HKDF(dek, "age
nt:" + agentId, agentNonce)
        Service->>Agent: Create Agent Token with Derived Gran
t<br/>agentGrant = wrapDEK(agentDEK, agentKey.publicKey)
    end

    rect rgb(232, 245, 233)
        Note over Agent: 4. AGENT ACCESSES ENCRYPTED DATA
        Agent->>Agent: Receive Grant via context.encryption
        Agent->>Agent: Decrypt DEK using derived key<br/>dek
= context.encryption.decryptDEK(grant)
        Agent->>DDC: Fetch encrypted data<br/>ciphertext = co
ntext.storage.get(cid)
        DDC-->>Agent: ciphertext
        Agent->>Agent: Decrypt data<br/>plaintext = context.e
ncryption.decrypt(ciphertext, dek)
    end
```

Token Specification

Extended Protobuf Definition

```
syntax = "proto3";

package ddc.auth;

option go_package = "cere.network/ddc-node/pb";

import "common/signature.proto";

// ============================================================
===================
// CORE TOKEN STRUCTURES (Extended from existing AuthToken)
// ============================================================
===================

message AuthToken {
  common.Signature signature = 1;
  Payload payload = 2;
}

message Payload {
  // Existing fields (backward compatible)
  optional AuthToken prev = 1;
  optional bytes subject = 2;
  optional bool canDelegate = 3;
  optional uint64 bucketId = 4;
  repeated Operation operations = 5;
  optional int64 expiresAt = 6;
  optional bytes pieceCid = 7;

  // NEW: Encryption extension (field numbers 10+ for extensi
on)
```

```
  optional EncryptionGrant encryptionGrant = 10;
}

enum Operation {
  UNKNOWN = 0;
  PUT = 1;
  GET = 2;
  DELETE = 3;
}

// ============================================================
===================
// ENCRYPTION GRANT STRUCTURES
// ============================================================
===================

message EncryptionGrant {
  // Unique identifier for this key (hash of DEK + creation n
once)
  bytes keyId = 1;

  // How the DEK is accessed
  oneof keyAccess {
    // Option A: DEK directly encrypted for recipient (non-re
vocable)
    EmbeddedKey embeddedKey = 2;

    // Option B: Reference to Key Escrow Service (revocable)
    KeyReference keyReference = 3;
  }

  // Constraints for key usage and delegation
  DelegationConstraints constraints = 4;

  // Proof of key derivation (for derived keys)
  optional KeyDerivationProof derivationProof = 5;
```

```
  // Encryption algorithm used for data
  EncryptionAlgorithm dataEncryptionAlgorithm = 6;
}

message EmbeddedKey {
  // DEK encrypted with recipient's X25519 public key via ECD
H
  bytes encryptedDataKey = 1;

  // Ephemeral public key used for ECDH (sender's ephemeral X
25519)
  bytes ephemeralPubKey = 2;

  // Nonce used for AES-GCM encryption of DEK
  bytes nonce = 3;
}

message KeyReference {
  // Key Escrow Service endpoint
  string kesEndpoint = 1;

  // Unique key identifier in KES
  bytes keyId = 2;

  // Policy ID for this specific grant (can be revoked)
  bytes policyId = 3;

  // Authorization proof (signed by key owner)
  common.Signature authorizationProof = 4;
}

message DelegationConstraints {
  // Maximum delegation depth from this point (0 = no further
delegation)
  uint32 maxDepth = 1;
```

```
  // Allowed operations for this grant and derived grants
  repeated Operation allowedOperations = 2;

  // Maximum TTL in seconds for derived grants
  int64 maxTTL = 3;

  // Specific agents allowed (empty = any agent under this se
rvice)
  repeated bytes allowedAgents = 4;

  // Whether key can be re-wrapped for other recipients
  bool allowRewrap = 5;
}

message KeyDerivationProof {
  // Parent key ID this was derived from
  bytes parentKeyId = 1;

  // Derivation algorithm
  KeyDerivationAlgorithm algorithm = 2;

  // Parameters used for derivation
  bytes derivationParams = 3;

  // Derived key public key (X25519)
  bytes derivedPubKey = 4;

  // Signature by parent key holder over derivation parameter
s
  common.Signature derivationSignature = 5;
}

enum KeyDerivationAlgorithm {
  KDA_UNKNOWN = 0;
  KDA_HKDF_SHA256 = 1;
```

```
  KDA_HKDF_SHA512 = 2;
}

enum EncryptionAlgorithm {
  EA_UNKNOWN = 0;
  EA_AES_256_GCM = 1;
  EA_CHACHA20_POLY1305 = 2;
}

// ============================================================
====================
// KEY ESCROW SERVICE MESSAGES
// ============================================================
====================

message DepositKeyRequest {
  // DEK encrypted with KES public key
  bytes encryptedKey = 1;

  // Ephemeral public key for ECDH
  bytes ephemeralPubKey = 2;

  // Owner's public key
  bytes ownerPubKey = 3;

  // Initial access policy
  AccessPolicy initialPolicy = 4;

  // Owner signature
  common.Signature signature = 5;
}

message DepositKeyResponse {
  // Assigned key ID
  bytes keyId = 1;
}
```

```
message AccessPolicy {
  repeated PolicyGrant grants = 1;
}

message PolicyGrant {
  // Unique policy ID (for revocation)
  bytes policyId = 1;

  // Grantee public key
  bytes subject = 2;

  // Allowed operations
  repeated Operation operations = 3;

  // Expiration
  int64 expiresAt = 4;

  // Constraints for further delegation
  DelegationConstraints constraints = 5;

  // Status
  GrantStatus status = 6;

  // Created timestamp
  int64 createdAt = 7;
}

enum GrantStatus {
  GS_UNKNOWN = 0;
  GS_ACTIVE = 1;
  GS_REVOKED = 2;
  GS_EXPIRED = 3;
}

message FetchKeyRequest {
```

```
  // Key ID
  bytes keyId = 1;

  // Policy ID
  bytes policyId = 2;

  // Requester public key
  bytes requesterPubKey = 3;

  // Timestamp (for replay protection)
  int64 timestamp = 4;

  // Signature proving requester identity
  common.Signature signature = 5;
}

message FetchKeyResponse {
  // DEK encrypted for requester's X25519 key
  bytes encryptedKey = 1;

  // Ephemeral public key for ECDH
  bytes ephemeralPubKey = 2;

  // Nonce
  bytes nonce = 3;

  // Current constraints
  DelegationConstraints constraints = 4;
}

message RevokeGrantRequest {
  bytes keyId = 1;
  bytes policyId = 2;
  bytes ownerPubKey = 3;
  common.Signature signature = 4;
}
```

```
message AddGrantRequest {
  bytes keyId = 1;
  PolicyGrant grant = 2;
  bytes ownerPubKey = 3;
  common.Signature signature = 4;
}

message ListGrantsRequest {
  bytes keyId = 1;
  bytes ownerPubKey = 2;
  common.Signature signature = 3;
}

message ListGrantsResponse {
  repeated PolicyGrant grants = 1;
}
```

Token Examples

Example 1: User → Agent Service (Embedded Key)

```
{
  "signature": {
    "algorithm": "ED_25519",
    "signer": "0x123...",
    "value": "sig_bytes..."
  },
  "payload": {
    "subject": "0xabc...",
    "bucketId": 123,
    "operations": ["GET"],
    "canDelegate": true,
    "expiresAt": 1735689600,
    "encryptionGrant": {
```

```
      "keyId": "key_abc123...",
      "embeddedKey": {
        "encryptedDataKey": "wrapped_dek_bytes...",
        "ephemeralPubKey": "ephemeral_x25519_pubkey...",
        "nonce": "random_12_bytes..."
      },
      "constraints": {
        "maxDepth": 1,
        "allowedOperations": ["GET"],
        "maxTTL": 86400,
        "allowRewrap": true
      },
      "dataEncryptionAlgorithm": "AES_256_GCM"
    }
  }
}
```

Example 2: User → Agent Service (Key Reference - Revocable)

```
{
  "signature": {
    "algorithm": "ED_25519",
    "signer": "0x123...",
    "value": "sig_bytes..."
  },
  "payload": {
    "subject": "0xabc...",
    "bucketId": 123,
    "operations": ["GET"],
    "canDelegate": true,
    "expiresAt": 1735689600,
    "encryptionGrant": {
      "keyId": "key_abc123...",
      "keyReference": {
        "kesEndpoint": "https://kes.cere.network",
```

```
        "keyId": "key_abc123...",
        "policyId": "policy_xyz789...",
        "authorizationProof": {
          "algorithm": "ED_25519",
          "signer": "0x123...",
          "value": "auth_sig..."
        }
      },
      "constraints": {
        "maxDepth": 1,
        "allowedOperations": ["GET"],
        "maxTTL": 86400
      }
    }
  }
}
```

Example 3: Agent Service → Agent (Derived Key)

```
{
  "signature": {
    "algorithm": "ED_25519",
    "signer": "0xabc...",
    "value": "sig_bytes..."
  },
  "payload": {
    "prev": { /* Parent token from user */ },
    "subject": null,
    "operations": ["GET"],
    "expiresAt": 1735603200,
    "encryptionGrant": {
      "keyId": "derived_key_agent1...",
      "embeddedKey": {
        "encryptedDataKey": "agent_wrapped_dek...",
        "ephemeralPubKey": "agent_ephemeral_pubkey...",
```

```
          "nonce": "agent_nonce..."
        },
        "constraints": {
          "maxDepth": 0,
          "allowedOperations": ["GET"],
          "maxTTL": 3600,
          "allowRewrap": false
        },
        "derivationProof": {
          "parentKeyId": "key_abc123...",
          "algorithm": "HKDF_SHA256",
          "derivationParams": "agent:agent1:nonce123",
          "derivedPubKey": "agent_derived_pubkey...",
          "derivationSignature": {
            "algorithm": "ED_25519",
            "signer": "0xabc...",
            "value": "derivation_sig..."
          }
        }
      }
    }
}
```

Key Management Options

Option A: Embedded Keys (Non-Revocable)

Flow:

```
flowchart LR
    User[👤 User] --> Encrypt["🔐 Encrypt DEK with<br/>recipi
ent pubkey"]
    Encrypt --> Embed["📦 Embed in token"]
    Embed --> Send["📤 Send token"]
```

```
    style User fill:#e3f2fd
    style Encrypt fill:#fff3e0
    style Embed fill:#f3e5f5
    style Send fill:#e8f5e9
```

Pros:

- Simple implementation
- Offline operation (no network calls)
- Fast (no KES latency)
- Decentralized (no central service)

Cons:

- Cannot revoke after token is issued
- Recipient can store and reuse key after token expiration
- No audit trail

Best For:

- Short-lived grants (< 24 hours)
- Low-sensitivity data
- Scenarios where revocation is not critical

Implementation:

```
class EmbeddedKeyGrant {
  async createGrant(
    dek: CryptoKey,
    recipientPubKey: Uint8Array,
    constraints: DelegationConstraints
  ): Promise<EncryptionGrant> {
    // Generate ephemeral X25519 keypair
    const ephemeralKeyPair = await crypto.subtle.generateKey(
      { name: 'X25519' },
      true,
```

```
      ['deriveBits']
    );

    // Derive shared secret via ECDH
    const sharedSecret = await crypto.subtle.deriveBits(
      { name: 'X25519', public: recipientPubKey },
      ephemeralKeyPair.privateKey,
      256
    );

    // Encrypt DEK with shared secret
    const nonce = crypto.getRandomValues(new Uint8Array(12));
    const wrappedDEK = await crypto.subtle.encrypt(
      { name: 'AES-GCM', iv: nonce },
      await crypto.subtle.importKey('raw', sharedSecret, 'AES
-GCM', false, ['encrypt']),
      await crypto.subtle.exportKey('raw', dek)
    );

    return {
      keyId: await this.computeKeyId(dek, nonce),
      embeddedKey: {
        encryptedDataKey: new Uint8Array(wrappedDEK),
        ephemeralPubKey: await crypto.subtle.exportKey('raw',
ephemeralKeyPair.publicKey),
        nonce: nonce,
      },
      constraints: constraints,
      dataEncryptionAlgorithm: 'AES_256_GCM',
    };
  }

  async decryptDEK(
    grant: EncryptionGrant,
    recipientPrivateKey: CryptoKey
  ): Promise<CryptoKey> {
```

```
    // Derive shared secret
    const sharedSecret = await crypto.subtle.deriveBits(
      { name: 'X25519', public: grant.embeddedKey.ephemeralPu
bKey },
      recipientPrivateKey,
      256
    );

    // Decrypt DEK
    const dekBytes = await crypto.subtle.decrypt(
      { name: 'AES-GCM', iv: grant.embeddedKey.nonce },
      await crypto.subtle.importKey('raw', sharedSecret, 'AES
-GCM', false, ['decrypt']),
      grant.embeddedKey.encryptedDataKey
    );

    return crypto.subtle.importKey('raw', dekBytes, 'AES-GC
M', true, ['encrypt', 'decrypt']);
  }
}
```

Option B: Key Escrow Service (Revocable)

Flow:

```
flowchart LR
    User[👤 User] --> Deposit["📥 Deposit DEK in KES"]
    Deposit --> Policy["📋 Create policy"]
    Policy --> Issue["📤 Issue token with<br/>key reference"]
    Policy --> Revoke["🚫 Revoke anytime"]

    style User fill:#e3f2fd
    style Deposit fill:#fff3e0
    style Policy fill:#f3e5f5
```

```
    style Issue fill:#e8f5e9
    style Revoke fill:#ffcdd2
```

Architecture:

```
flowchart TB
    subgraph KES["🏛️ KEY ESCROW SERVICE (KES)"]
        subgraph KeyStore["🔐 Key Store (HSM-backed)"]
            Key001["Key 001<br/>(0x123)"]
            Key002["Key 002<br/>(0x456)"]
            Key003["Key 003<br/>(0x789)"]
        end

        subgraph PolicyStore["📋 Policy Store"]
            Policy1["Key 001:<br/>├─ Policy A: subject=0xabc,
ops=[GET]<br/>│  expires=2024-12-31, status=ACTIVE<br/>└─ Pol
icy B: subject=0xdef, ops=[GET]<br/>   expires=2024-06-30, st
atus=REVOKED"]
        end

        subgraph AuditLog["📜 Audit Log"]
            Log1["2024-01-15 10:00:00 DEPOSIT key=001 owner=0
x123"]
            Log2["2024-01-15 10:01:00 GRANT key=001 subject=0
xabc"]
            Log3["2024-01-15 11:00:00 FETCH key=001 requester
=0xabc"]
            Log4["2024-01-16 09:00:00 REVOKE key=001 policy=
B"]
        end
    end

    style KES fill:#fafafa
    style KeyStore fill:#e3f2fd
```

```
    style PolicyStore fill:#fff3e0
    style AuditLog fill:#f3e5f5
```

Pros:

- Instant revocation
- Full audit trail
- Centralized policy management
- Can update constraints without re-issuing tokens

Cons:

- Requires KES availability
- Single point of failure (mitigated with replication)
- Network latency for key fetch
- Operational overhead

Best For:

- Long-lived grants (> 24 hours)
- Sensitive data
- Enterprise/compliance requirements
- Scenarios requiring revocation

API Specification:

```
interface KeyEscrowService {
  /**
   * Deposit a DEK with initial access policy
   */
  depositKey(request: {
    encryptedKey: Uint8Array;       // DEK encrypted with KES
public key
    ephemeralPubKey: Uint8Array;    // For ECDH
    ownerPubKey: Uint8Array;        // Owner's Ed25519 public
key
```

```
    initialPolicy?: AccessPolicy;   // Optional initial grant
s
    signature: Signature;           // Owner's signature
  }): Promise<{ keyId: Uint8Array }>;

  /**
   * Add a new grant to existing key
   */
  addGrant(request: {
    keyId: Uint8Array;
    grant: PolicyGrant;
    ownerPubKey: Uint8Array;
    signature: Signature;
  }): Promise<{ policyId: Uint8Array }>;

  /**
   * Revoke an existing grant (immediate effect)
   */
  revokeGrant(request: {
    keyId: Uint8Array;
    policyId: Uint8Array;
    ownerPubKey: Uint8Array;
    signature: Signature;
  }): Promise<void>;

  /**
   * Update grant constraints
   */
  updateGrant(request: {
    keyId: Uint8Array;
    policyId: Uint8Array;
    updates: Partial<PolicyGrant>;
    ownerPubKey: Uint8Array;
    signature: Signature;
  }): Promise<void>;
```

```
  /**
   * Fetch DEK (called by grantees)
   */
  fetchKey(request: {
    keyId: Uint8Array;
    policyId: Uint8Array;
    requesterPubKey: Uint8Array;
    timestamp: number;
    signature: Signature;
  }): Promise<{
    encryptedKey: Uint8Array;
    ephemeralPubKey: Uint8Array;
    nonce: Uint8Array;
    constraints: DelegationConstraints;
  }>;

  /**
   * List all grants for a key (owner only)
   */
  listGrants(request: {
    keyId: Uint8Array;
    ownerPubKey: Uint8Array;
    signature: Signature;
  }): Promise<{ grants: PolicyGrant[] }>;

  /**
   * Get audit log for a key (owner only)
   */
  getAuditLog(request: {
    keyId: Uint8Array;
    ownerPubKey: Uint8Array;
    signature: Signature;
    fromTime?: number;
    toTime?: number;
```

```
  }): Promise<{ entries: AuditEntry[] }>;
}
```

Option C: On-Chain Policy Registry

Flow:

```
flowchart LR
    User[👤 User] --> Submit["📝 Submit grant<br/>to pallet"]
    Submit --> OnChain["⛓ On-chain storage"]
    OnChain --> Query["🔍 Storage nodes<br/>query chain"]

    style User fill:#e3f2fd
    style Submit fill:#fff3e0
    style OnChain fill:#f3e5f5
    style Query fill:#e8f5e9
```

Pallet Extension:

```
// Extend DDC pallet with encryption grants
#[pallet::storage]
pub type EncryptionGrants<T: Config> = StorageDoubleMap<
    _,
    Blake2_128Concat, KeyId,
    Blake2_128Concat, PolicyId,
    EncryptionGrantInfo<T>,
    OptionQuery,
>;

#[derive(Encode, Decode, Clone, PartialEq, Eq, RuntimeDebug,
TypeInfo)]
pub struct EncryptionGrantInfo<T: Config> {
    pub owner: T::AccountId,
    pub subject: T::AccountId,
    pub encrypted_dek: BoundedVec<u8, MaxDekSize>,
```

```
    pub ephemeral_pubkey: [u8; 32],
    pub operations: Operations,
    pub expires_at: T::BlockNumber,
    pub constraints: DelegationConstraints,
    pub status: GrantStatus,
    pub created_at: T::BlockNumber,
}

#[pallet::call]
impl<T: Config> Pallet<T> {
    #[pallet::weight(10_000)]
    pub fn create_encryption_grant(
        origin: OriginFor<T>,
        key_id: KeyId,
        subject: T::AccountId,
        encrypted_dek: BoundedVec<u8, MaxDekSize>,
        ephemeral_pubkey: [u8; 32],
        operations: Operations,
        expires_at: T::BlockNumber,
        constraints: DelegationConstraints,
    ) -> DispatchResult {
        let who = ensure_signed(origin)?;

        // Verify caller is key owner
        ensure!(Self::is_key_owner(&who, &key_id), Error::<T
>::NotKeyOwner);

        // Generate policy ID
        let policy_id = Self::generate_policy_id(&key_id, &su
bject);

        // Store grant
        EncryptionGrants::<T>::insert(key_id, policy_id, Encr
yptionGrantInfo {
            owner: who.clone(),
            subject,
```

```
            encrypted_dek,
            ephemeral_pubkey,
            operations,
            expires_at,
            constraints,
            status: GrantStatus::Active,
            created_at: <frame_system::Pallet<T>>::block_numb
er(),
        });

        Self::deposit_event(Event::EncryptionGrantCreated { k
ey_id, policy_id });
        Ok(())
    }

    #[pallet::weight(5_000)]
    pub fn revoke_encryption_grant(
        origin: OriginFor<T>,
        key_id: KeyId,
        policy_id: PolicyId,
    ) -> DispatchResult {
        let who = ensure_signed(origin)?;

        // Verify caller is key owner
        ensure!(Self::is_key_owner(&who, &key_id), Error::<T
>::NotKeyOwner);

        // Update status to revoked
        EncryptionGrants::<T>::mutate(key_id, policy_id, |may
be_grant| {
            if let Some(grant) = maybe_grant {
                grant.status = GrantStatus::Revoked;
            }
        });

        Self::deposit_event(Event::EncryptionGrantRevoked { k
```

```
ey_id, policy_id });
        Ok(())
    }
}
```

Pros:

- Fully decentralized
- Immutable audit trail
- No central point of failure
- Cryptographically verifiable

Cons:

- Transaction costs (~0.01 CERE per operation)
- Latency (~6 seconds for finality)
- On-chain storage costs
- Less flexible updates

Best For:

- High-value assets
- Regulatory requirements for immutability
- Cross-chain interoperability
- Trustless environments

Option D: Hybrid (Recommended)

Combine the best of each approach:

```
interface HybridKeyManager {
  /**
   * Create grant with appropriate strategy based on requirem
ents
   */
```

```
  createGrant(options: {
    keyId: string;
    subject: string;

    // Strategy selection
    strategy: 'embedded' | 'kes' | 'on-chain' | 'auto';

    // Constraints
    expiresIn: number;
    operations: Operation[];
    constraints: DelegationConstraints;

    // Hints for 'auto' strategy
    hints?: {
      sensitivityLevel?: 'low' | 'medium' | 'high';
      revocationRequired?: boolean;
      auditRequired?: boolean;
    };
  }): Promise<AuthToken>;
}

// Auto-strategy selection logic
function selectStrategy(hints: StrategyHints): Strategy {
  if (hints.sensitivityLevel === 'high' || hints.auditRequire
d) {
    return 'on-chain';
  }

  if (hints.revocationRequired) {
    return 'kes';
  }

  if (hints.expiresIn < 24 * 60 * 60 * 1000) { // < 24 hours
    return 'embedded';
  }
```

```
  return 'kes'; // Default for longer-lived grants
}
```

Decision Matrix:

Requirement	Embedded	KES	On-Chain
Short-lived (<24h)	✅ Best	⚠️ OK	❌ Slow
Revocable	❌ No	✅ Best	✅ Yes
Audit trail	❌ No	✅ Best	✅ Immutable
High availability	✅ Offline	⚠️ Service SLA	✅ Blockchain
Low latency	✅ Best	⚠️ Network	❌ ~6s
Decentralized	✅ Yes	❌ No	✅ Best
Cost	✅ Free	⚠️ Service fee	⚠️ Tx fee

SDK Integration

TypeScript SDK

```
import {
  DdcClient,
  AuthToken,
  EncryptedDataManager,
  KeyEscrowService,
  Operation
} from '@cere-ddc-sdk/ddc-client';

// Initialize
const userSigner = new UriSigner('user_mnemonic...');
const ddc = await DdcClient.create(userSigner, TESTNET);
const kes = new KeyEscrowService({ endpoint: 'https://kes.cer
e.network' });
const edm = new EncryptedDataManager(ddc, kes, userSigner);

// ============================================================
```

```
==
// 1. Upload Encrypted Data
// ============================================================
==
const sensitiveData = new TextEncoder().encode('My secret dat
a');

const { cid, keyId } = await edm.uploadEncrypted({
  data: sensitiveData,
  bucketId: 123n,
  // Optional initial grants
  grants: [{
    subject: '0xDataServiceA',
    operations: [Operation.GET],
    expiresIn: 7 * 24 * 60 * 60 * 1000, // 7 days
    strategy: 'kes', // Revocable
    constraints: {
      maxDepth: 1,
      maxAgentTTL: 3600,
    },
  }],
});

console.log('Encrypted data uploaded:', cid);
console.log('Key registered:', keyId);

// ============================================================
==
// 2. Add Grant for New Data Service
// ============================================================
==
const grantToken = await edm.addGrant({
  keyId,
  subject: '0xDataServiceB',
  operations: [Operation.GET],
  expiresIn: 30 * 24 * 60 * 60 * 1000, // 30 days
```

```
  strategy: 'kes',
  constraints: {
    maxDepth: 0, // Cannot delegate to agents
  },
});

console.log('Grant created for Service B:', grantToken.toStri
ng());

// ============================================================
==
// 3. List Active Grants
// ============================================================
==
const grants = await edm.listGrants(keyId);
console.log('Active grants:', grants);
// [
//   { policyId: 'abc', subject: '0xDataServiceA', status: 'A
CTIVE', ... },
//   { policyId: 'def', subject: '0xDataServiceB', status: 'A
CTIVE', ... }
// ]

// ============================================================
==
// 4. Revoke Grant
// ============================================================
==
await edm.revokeGrant({
  keyId,
  policyId: grants[0].policyId,
});

console.log('Grant revoked for Service A');

// ============================================================
```

```
==
// 5. Update Grant Constraints
// ============================================================
==
await edm.updateGrant({
  keyId,
  policyId: grants[1].policyId,
  updates: {
    expiresIn: 60 * 24 * 60 * 60 * 1000, // Extend to 60 days
  },
});

console.log('Grant extended for Service B');

// ============================================================
==
// 6. Get Audit Log
// ============================================================
==
const auditLog = await edm.getAuditLog(keyId);
console.log('Key access history:', auditLog);
// [
//   { timestamp: ..., action: 'DEPOSIT', owner: '0x123' },
//   { timestamp: ..., action: 'GRANT', subject: '0xServiceA'
},
//   { timestamp: ..., action: 'FETCH', requester: '0xService
A' },
//   { timestamp: ..., action: 'REVOKE', policyId: 'abc' },
// ]
```

Agent Service SDK

```
import { DataServiceClient, KeyDerivationService } from '@cer
e-ddc-sdk/data-service';
```

```
// Data Service receives grant from user
const userGrantToken = AuthToken.from(receivedTokenString);

// Initialize
const serviceSigner = new UriSigner('service_mnemonic...');
const kds = new KeyDerivationService(serviceSigner);

// ============================================================
==
// 1. Decrypt User's DEK
// ============================================================
==
const dek = await kds.decryptDEK(userGrantToken.encryptionGra
nt);

// ============================================================
==
// 2. Derive Key for Specific Agent
// ============================================================
==
const agentId = 'sentiment-analyzer';
const agentGrant = await kds.deriveAgentGrant({
  parentGrant: userGrantToken.encryptionGrant,
  agentId: agentId,
  expiresIn: 3600 * 1000, // 1 hour
});

// ============================================================
==
// 3. Create Agent Token
// ============================================================
==
const agentToken = new AuthToken({
  prev: userGrantToken,
  operations: [Operation.GET],
  expiresIn: 3600 * 1000,
```

```
  encryptionGrant: agentGrant,
});

await agentToken.sign(serviceSigner);

// Pass token to agent via execution context
await orchestrator.executeAgent({
  agentId: agentId,
  event: { payload: { cid: 'data_cid...' } },
  context: {
    encryptionGrant: agentGrant,
    authToken: agentToken.toString(),
  },
});
```

Agent Context API

```
// Agent code running in V8 isolate
async function handle(event: any, ctx: any) {
  // ==========================================================
====
  // 1. Get Encryption Grant from Context
  // ==========================================================
====
  const grant = ctx.encryption.getGrant();

  if (!grant) {
    ctx.log('No encryption grant provided');
    return { error: 'Missing encryption grant' };
  }

  // ==========================================================
====
  // 2. Decrypt DEK Using Derived Key
  // ==========================================================
```

```
====
  const dek = await ctx.encryption.decryptDEK(grant);

  // ==========================================================
====
  // 3. Fetch Encrypted Data
  // ==========================================================
====
  const encryptedData = await ctx.storage.get(event.payload.c
id);

  // ==========================================================
====
  // 4. Decrypt Data
  // ==========================================================
====
  const plainData = await ctx.encryption.decrypt(encryptedDat
a, dek);

  // ==========================================================
====
  // 5. Process Data
  // ==========================================================
====
  const result = await ctx.models.infer('gpt-4', {
    prompt: 'Analyze this data',
    data: new TextDecoder().decode(plainData),
  });

  return { analysis: result };
}
```

Storage Node Verification

```
// Extended gater.go for encryption grant verification
func (g *gater) expandAndVerifyToken(token *pb.AuthToken, exp
anded *ExpandedToken, depth int) error {
    // ... existing verification ...

    // Verify encryption grant if present
    if token.Payload.EncryptionGrant != nil {
        if err := g.verifyEncryptionGrant(token, expanded, de
pth); err != nil {
            return fmt.Errorf("encryption grant verification
failed: %w", err)
        }
    }

    // Continue to parent token
    if token.Payload.Prev != nil {
        return g.expandAndVerifyToken(token.Payload.Prev, exp
anded, depth+1)
    }

    return nil
}

func (g *gater) verifyEncryptionGrant(token *pb.AuthToken, ex
panded *ExpandedToken, depth int) error {
    grant := token.Payload.EncryptionGrant

    // 1. Verify constraints
    if grant.Constraints.MaxDepth < uint32(depth) {
        return ErrDelegationDepthExceeded
    }

    // 2. Verify operations subset
    if !operationsSubset(grant.Constraints.AllowedOperations,
token.Payload.Operations) {
```

```
        return ErrInvalidGrantOperations
    }

    // 3. Verify TTL constraint
    tokenTTL := token.Payload.ExpiresAt - time.Now().Unix()
    if tokenTTL > grant.Constraints.MaxTTL {
        return ErrGrantTTLExceeded
    }

    // 4. Verify key reference if using KES
    if grant.KeyReference != nil {
        if err := g.verifyKeyReference(grant.KeyReference); e
rr != nil {
            return err
        }
    }

    // 5. Verify derivation proof if derived key
    if grant.DerivationProof != nil {
        if err := g.verifyDerivationProof(grant.DerivationPro
of, expanded.parentKeyId); err != nil {
            return err
        }
    }

    // 6. Store key ID for activity capture
    expanded.keyId = grant.KeyId

    return nil
}

func (g *gater) verifyKeyReference(ref *pb.KeyReference) erro
r {
    // Query KES to verify policy is active
    client := g.kesClients[ref.KesEndpoint]
    if client == nil {
```

```
        return ErrUnknownKES
    }

    status, err := client.GetPolicyStatus(ref.KeyId, ref.Poli
cyId)
    if err != nil {
        return fmt.Errorf("KES query failed: %w", err)
    }

    if status != pb.GrantStatus_GS_ACTIVE {
        return ErrGrantRevoked
    }

    return nil
}

func (g *gater) verifyDerivationProof(proof *pb.KeyDerivation
Proof, expectedParentKeyId []byte) error {
    // 1. Verify parent key ID matches
    if !bytes.Equal(proof.ParentKeyId, expectedParentKeyId) {
        return ErrInvalidDerivationParent
    }

    // 2. Verify derivation signature
    message := append(proof.DerivedPubKey, proof.DerivationPa
rams...)
    valid, err := pb.VerifySignatureBytes(
        proof.DerivationSignature.Signer,
        message,
        proof.DerivationSignature.Value,
        proof.DerivationSignature.Algorithm,
    )
    if err != nil || !valid {
        return ErrInvalidDerivationSignature
    }
```

```
    return nil
}
```

Implementation Phases

Phase 1: Core Encryption (4 weeks)

Deliverables:
- [ ] `Cipher` interface in SDK
- [ ] AES-256-GCM implementation
- [ ] Encrypt/decrypt in `upload()` / `download()`
- [ ] CLI flags: `--encrypt`, `--key`
- [ ] Unit and integration tests

Files Modified:
- `packages/ddc/src/encryption/Cipher.ts`
- `packages/ddc/src/encryption/AesGcmCipher.ts`
- `packages/ddc/src/DdcClient.ts`
- `packages/cli/src/commands/upload.ts`
- `packages/cli/src/commands/download.ts`

Phase 2: Encryption Grant in Token (3 weeks)

Deliverables:
- [ ] Extended protobuf definitions
- [ ] `EncryptionGrant` creation in SDK
- [ ] `EmbeddedKey` implementation
- [ ] Token serialization with grants
- [ ] Update storage node for grant verification

Files Modified:
- `packages/ddc/protos/auth_token.proto`
- `packages/ddc/src/auth/AuthToken.ts`
- `packages/ddc/src/auth/EncryptionGrant.ts`
- `internal/storage/auth/gater.go`

Phase 3: Key Derivation (2 weeks)

Deliverables:
- [ ] HKDF implementation
- [ ] Agent key derivation
- [ ] `KeyDerivationProof` generation and verification
- [ ] Data service SDK support

Files Modified:
- `packages/ddc/src/encryption/KeyDerivation.ts`
- `packages/ddc/src/encryption/DerivationProof.ts`

Phase 4: Key Escrow Service (4 weeks)

Deliverables:
- [ ] KES API specification
- [ ] KES server implementation
- [ ] SDK client for KES
- [ ] Grant lifecycle management (add/revoke/update)
- [ ] Audit logging

New Components:
- `services/kes/` - Key Escrow Service
- `packages/ddc/src/kes/KeyEscrowClient.ts`

Phase 5: Agent Runtime Integration (3 weeks)

Deliverables:
- [ ] `context.encryption` API in agent runtime
- [ ] `context.secureStorage` wrapper
- [ ] Grant injection via execution context
- [ ] V8 crypto bindings

Files Modified:
- `internal/compute/agentruntime/context/encryption.go`
- `internal/compute/agentruntime/context/builder.go`
- `internal/compute/agentruntime/shared/types.go`

Phase 6: On-Chain Registry (Optional, 4 weeks)

Deliverables:
- [ ] Pallet extension for encryption grants

- [ ] On-chain grant management
- [ ] Storage node blockchain query integration
- [ ] Migration tools

Files Modified:

- `pallets/ddc/src/lib.rs`
- `internal/storage/blockchain/indexer.go`

Security Considerations

Threat Model

Threat	Mitigation
Compromised storage node	Data encrypted client-side; nodes never see plaintext
Key theft	Keys never transmitted in plaintext; always wrapped via ECDH
Replay attacks	Timestamps and nonces in key fetch requests
Privilege escalation	Constraint validation at each delegation level
Unauthorized revocation	Owner signature required for all KES operations
KES compromise	DEKs encrypted at rest; HSM for key storage
Agent key leakage	Derived keys are short-lived and scoped

Cryptographic Choices

Component	Algorithm	Rationale
Data encryption	AES-256-GCM	AEAD, hardware acceleration
Key exchange	X25519 (ECDH)	Fast, secure, compatible with Ed25519
Key derivation	HKDF-SHA256	RFC 5869, deterministic
Signing	Ed25519	Fast, small signatures
Key ID	BLAKE3	Fast, collision resistant

Key Rotation

```
interface KeyRotationPolicy {
  // Maximum key age before rotation required
  maxKeyAge: Duration;

  // Grace period for old keys after rotation
  gracePeriod: Duration;

  // Automatic rotation trigger
  autoRotate: boolean;

  // Notification webhook for rotation events
  notifyUrl?: string;
}

async function rotateKey(keyId: string, policy: KeyRotationPo
licy): Promise<string> {
  // 1. Generate new DEK
  const newDEK = await generateDEK();

  // 2. Fetch current grants
  const grants = await kes.listGrants(keyId);
  const activeGrants = grants.filter(g => g.status === 'ACTIV
E');

  // 3. Re-wrap DEK for all active grantees
  const newGrants = await Promise.all(
    activeGrants.map(async (grant) => ({
      ...grant,
      encryptedDataKey: await wrapDEK(newDEK, grant.subject),
    }))
  );

  // 4. Deposit new key with migrated grants
  const newKeyId = await kes.depositKey({
    key: newDEK,
```

```
    owner: ownerPubKey,
    initialPolicy: { grants: newGrants },
  });

  // 5. Re-encrypt data (if required)
  if (policy.reEncryptData) {
    await reEncryptData(keyId, newKeyId);
  }

  // 6. Schedule old key deletion after grace period
  await kes.scheduleKeyDeletion(keyId, policy.gracePeriod);

  return newKeyId;
}
```

Consequences

Positive

1. **Privacy by Default**: User data encrypted at rest; storage nodes cannot read content
2. **Flexible Access Control**: Multiple key management strategies for different requirements
3. **Hierarchical Delegation**: Clean user → service → agent key hierarchy
4. **Revocability**: Can revoke access to existing encrypted data (with KES)
5. **Audit Trail**: Full visibility into key access patterns
6. **Backward Compatibility**: Encryption is optional; existing tokens continue to work
7. **Unified Pattern**: Same authorization model for storage and compute

Negative

1. **Complexity**: Additional cryptographic operations increase implementation complexity
2. **Performance**: Encryption adds ~5% overhead; KES adds network latency
3. **Key Management**: Users must securely store their master keys
4. **Recovery**: Lost DEK means data is permanently inaccessible
5. **KES Dependency**: Revocable grants require KES availability

Risks

Risk	Likelihood	Impact	Mitigation
KES downtime	Medium	High	Replicated deployment; fallback to embedded keys
Key loss	Low	Critical	Key backup guidance; recovery phrases
Cryptographic vulnerability	Low	Critical	Use standard algorithms; security audits
Performance degradation	Medium	Medium	Hardware acceleration; caching

---

References

1. [ADR] Authentication and authorization - Existing DDC auth system
2. RFC 5869 - HKDF (HMAC-based Key Derivation Function)
3. RFC 7748 - Elliptic Curves for Security (X25519)
4. RFC 5116 - AEAD (AES-GCM)

Appendix A: Cryptographic Primitives

A.1 Key Generation

```
// Generate Ed25519 signing key
async function generateSigningKey(): Promise<CryptoKeyPair> {
```

```
  return crypto.subtle.generateKey(
    { name: 'Ed25519' },
    true,
    ['sign', 'verify']
  );
}

// Derive X25519 encryption key from Ed25519
async function deriveEncryptionKey(ed25519Private: CryptoKe
y): Promise<CryptoKeyPair> {
  // Ed25519 private key can be converted to X25519
  // See: https://doc.libsodium.org/advanced/ed25519-curve255
19
  const rawKey = await crypto.subtle.exportKey('raw', ed25519
Private);
  const x25519Private = ed25519ToX25519(rawKey);
  return crypto.subtle.importKey('raw', x25519Private, 'X2551
9', true, ['deriveBits']);
}

// Generate DEK
async function generateDEK(): Promise<CryptoKey> {
  return crypto.subtle.generateKey(
    { name: 'AES-GCM', length: 256 },
    true,
    ['encrypt', 'decrypt']
  );
}
```

A.2 ECDH Key Exchange

```
async function wrapDEK(
  dek: CryptoKey,
  recipientPubKey: Uint8Array
): Promise<{ wrapped: Uint8Array; ephemeralPub: Uint8Array; n
```

```
once: Uint8Array }> {
  // Generate ephemeral keypair
  const ephemeral = await crypto.subtle.generateKey(
    { name: 'X25519' },
    true,
    ['deriveBits']
  );

  // ECDH to derive shared secret
  const recipientKey = await crypto.subtle.importKey(
    'raw',
    recipientPubKey,
    { name: 'X25519' },
    false,
    []
  );

  const sharedSecret = await crypto.subtle.deriveBits(
    { name: 'X25519', public: recipientKey },
    ephemeral.privateKey,
    256
  );

  // Derive wrapping key via HKDF
  const wrappingKey = await crypto.subtle.importKey(
    'raw',
    sharedSecret,
    'AES-GCM',
    false,
    ['encrypt']
  );

  // Wrap DEK
  const nonce = crypto.getRandomValues(new Uint8Array(12));
  const dekBytes = await crypto.subtle.exportKey('raw', dek);
  const wrapped = await crypto.subtle.encrypt(
```

```
    { name: 'AES-GCM', iv: nonce },
    wrappingKey,
    dekBytes
  );

  return {
    wrapped: new Uint8Array(wrapped),
    ephemeralPub: new Uint8Array(await crypto.subtle.exportKe
y('raw', ephemeral.publicKey)),
    nonce: nonce,
  };
}
```

A.3 HKDF Key Derivation

```
async function deriveAgentKey(
  masterKey: CryptoKey,
  agentId: string,
  nonce: Uint8Array
): Promise<CryptoKey> {
  const masterKeyBytes = await crypto.subtle.exportKey('raw',
masterKey);

  // HKDF-SHA256
  const hkdfKey = await crypto.subtle.importKey(
    'raw',
    masterKeyBytes,
    'HKDF',
    false,
    ['deriveBits', 'deriveKey']
  );

  const info = new TextEncoder().encode(`agent:${agentId}`);

  return crypto.subtle.deriveKey(
```

```
    {
      name: 'HKDF',
      hash: 'SHA-256',
      salt: nonce,
      info: info,
    },
    hkdfKey,
    { name: 'AES-GCM', length: 256 },
    true,
    ['encrypt', 'decrypt']
  );
}
```

Appendix B: Error Codes

Code	Name	Description
`E_GRANT_EXPIRED`	Grant Expired	Encryption grant has expired
`E_GRANT_REVOKED`	Grant Revoked	Encryption grant was revoked
`E_DEPTH_EXCEEDED`	Delegation Depth Exceeded	Too many delegation levels
`E_TTL_EXCEEDED`	TTL Exceeded	Token TTL exceeds grant constraint
`E_INVALID_DERIVATION`	Invalid Derivation	Key derivation proof is invalid
`E_KEY_NOT_FOUND`	Key Not Found	Key ID not found in KES
`E_POLICY_NOT_FOUND`	Policy Not Found	Policy ID not found
`E_UNAUTHORIZED`	Unauthorized	Requester not authorized for key
`E_INVALID_SIGNATURE`	Invalid Signature	Signature verification failed
`E_KES_UNAVAILABLE`	KES Unavailable	Key Escrow Service unreachable